IdeaDNA

Technical Architecture Document (TAD)

"System Design & Architecture Blueprint"

Document Type:	Technical Architecture Document
Version:	v1.0
Date:	June 2026
Author:	Platform Developer
Status:	Approved for Development

■ Table of Contents

1. Architecture Overview

IdeaDNA follows a modern full-stack architecture with clear separation of concerns. The system is built on a micro-frontend + modular backend pattern, with AI services orchestrated through LangChain. The architecture prioritizes: Cost efficiency (lazy loading), Type safety (TypeScript + tRPC), and Developer experience (Prisma ORM, Next.js App Router).

Architecture Principles

- Lazy Loading: Heavy assets (blueprints, mockups) generated only on-demand
- Type Safety: End-to-end TypeScript from database to UI
- Modularity: Each feature is self-contained with clear interfaces
- Scalability: Stateless API design with horizontal scaling capability
- Cost Optimization: Caching, rate limiting, and intelligent resource allocation

2. System Components & Services

Client Layer (Frontend)

Next.js 14 App Router, TypeScript, Tailwind CSS, React Query, Zustand. Responsible for: UI rendering, state management, user interactions, API consumption.

API Gateway Layer

Next.js API Routes + tRPC routers. Responsible for: Request routing, auth validation, rate limiting, request/response transformation.

Application Layer

Business logic services: IdeaGeneratorService, EvolutionEngineService, CodeArchaeologyService, BlueprintService, MockupService, ScoringService.

AI Orchestration Layer

LangChain AgentExecutor, SequentialChain, OpenAI API integration. Responsible for: Multi-agent pipelines, prompt management, output parsing.

Data Layer

PostgreSQL (primary), Redis (cache), Pinecone/pgvector (vector search). Responsible for: Persistent storage, session management, semantic search.

External Services

GitHub API, OpenAI API, Clerk Auth, Vercel Hosting, Railway Database.

3. Data Flow & Integration

Primary Flow: Idea Generation

- 1. User → Frontend: Enters problem + selects platform + confirms profile
- 2. Frontend → API Gateway: POST /api/generator/full (with user token)
- 3. API Gateway → Auth Service: Validate JWT via Clerk
- 4. API Gateway → IdeaGeneratorService: Forward request
- 5. IdeaGeneratorService → RetrieverAgent: Query curated database (Prisma + pgvector)
- 6. RetrieverAgent → PostgreSQL: Semantic search with embeddings
- 7. RetrieverAgent → UpgradeAgent: Personalize retrieved ideas
- 8. UpgradeAgent → OpenAI API: LLM call for idea upgrade
- 9. ScorerAgent → OpenAI API: LLM call for scoring
- 10. IdeaGeneratorService → PostgreSQL: Save ideas to DB
- 11. API Gateway → Frontend: Return 5-8 idea cards
- 12. User → Frontend: Selects one idea
- 13. Frontend → API Gateway: POST /api/ideas/:id/select
- 14. API Gateway → BlueprintService: Trigger lazy-loaded generation
- 15. BlueprintService → BlueprintArchitectAgent: Generate 7 sections
- 16. BlueprintArchitectAgent → OpenAI API: LLM call for blueprint
- 17. BlueprintService → PostgreSQL: Save blueprint
- 18. API Gateway → Frontend: Return blueprint data

4. Database Architecture

PostgreSQL 15 with Prisma ORM. Primary database handles all relational data. pgvector extension for vector embeddings. Redis for caching and sessions. Connection pooling via PgBouncer.

4.1 Entity Relationship Overview

- User (1) → Project (N): One user has many projects
- Project (1) → Idea (N): One project has many generated ideas
- Project (1) → Idea (1): One project has one selected idea
- Idea (1) → Blueprint (1): One idea has one blueprint (lazy-loaded)
- Idea (1) → UiMockup (1): One idea has one UI mockup (lazy-loaded)
- Project (1) → EvolutionTree (1): Evolution mode projects have one tree
- Project (1) → ArchaeologyReport (1): Archaeology mode projects have one report
- Project (N) → Project (N): Self-referencing for evolution lineage

4.2 Indexing Strategy

- userId (Project): For user's project list queries
- projectId (Idea): For project's ideas queries
- selectedIdeaId (Project): For selected idea lookup
- mode + status (Project): For dashboard filtering
- platform + complexity (CuratedIdea): For filtered retrieval
- embedding (CuratedIdea): GIN index for vector similarity search
- createdAt (all tables): For sorting and pagination

5. AI/ML Architecture

LangChain-based multi-agent orchestration system. Each mode has a dedicated SequentialChain. Agents communicate through structured JSON outputs. Temperature tuning per agent role.

5.1 Agent Configuration

Agent	Temperature	Model	Purpose	Output Format
InputParser	0.3	GPT-4	Extract structured info from raw input	JSON
Retriever	N/A	Embeddings	Semantic search in curated DB	Array of IDs
UpgradeAgent	0.6	GPT-4	Personalize ideas for user	JSON
CriticAgent	0.3	GPT-4	Analyze weaknesses objectively	JSON
EvolverAgent	0.6	GPT-4	Generate mutated versions	JSON
ScorerAgent	0.2	GPT-4	Objective scoring	JSON
BlueprintArchitect	0.4	GPT-4	Generate project blueprint	JSON
UIDesigner	0.5	GPT-4	Generate HTML/CSS mockup	HTML
CodeParser	0.3	GPT-4	Parse and analyze code	JSON
PatternDetector	0.3	GPT-4	Identify design patterns	JSON
IdeaHistorian	0.5	GPT-4	Reconstruct evolution timeline	JSON
DevPsychologist	0.4	GPT-4	Infer developer thinking	JSON

6. API Design

6.1 REST API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	User registration via Clerk
POST	/api/auth/login	Public	User login via Clerk
GET	/api/user/profile	Required	Get current user profile
PUT	/api/user/profile	Required	Update user profile
GET	/api/projects	Required	List all user projects
POST	/api/projects	Required	Create new project
GET	/api/projects/:id	Required	Get project with ideas
POST	/api/projects/:id/generate	Required	Generate ideas (Generator mode)
POST	/api/projects/:id/evolve	Required	Evolve idea (Evolution mode)
POST	/api/projects/:id/analyze	Required	Analyze code (Archaeology mode)
POST	/api/ideas/:id/select	Required	Select idea → trigger deep-dive
GET	/api/ideas/:id/blueprint	Required	Get blueprint (lazy-loaded)
GET	/api/ideas/:id/mockup	Required	Get UI mockup (lazy-loaded)
POST	/api/ideas/:id/evolve-further	Required	Evolve selected idea further
GET	/api/export/blueprint/:id	Required	Export blueprint as Markdown/PDF
GET	/api/export/mockup/:id	Required	Export mockup as HTML
GET	/api/curated/platforms	Public	Get all platform categories
GET	/api/curated/trending	Public	Get trending ideas
POST	/api/search/similar	Required	Find similar past projects

7. Frontend Architecture

Next.js 14 with App Router. Component-based architecture with clear separation. State management via Zustand (client) + React Query (server). Design system built on Tailwind CSS with custom theme tokens.

7.1 Directory Structure

```
src/ ■■■ app/ # Next.js App Router ■ ■■■ (auth)/ # Auth group (login, register) ■ ■■■ (dashboard)/ # Dashboard group ■ ■ ■■■ page.tsx # Main dashboard ■ ■ ■■■ projects/ # Project list ■ ■ ■■■ settings/ # User settings ■ ■■■ generate/ # Idea Generator mode ■ ■■■ evolve/ # Evolution Engine mode ■ ■■■ analyze/ # Code Archaeology mode ■ ■■■ api/ # API Routes ■■■ components/ ■ ■■■ ui/ # Reusable UI primitives ■ ■■■ layout/ # Layout components ■ ■■■ dna-graph/ # Evolution tree (React Flow) ■ ■■■ idea-cards/ # Idea card components ■ ■■■ blueprint/ # Blueprint renderer ■ ■■■ mockup-preview/ # UI mockup iframe ■ ■■■ scoring/ # Radar chart (D3.js) ■■■ lib/ ■ ■■■ langchain/ # Agent definitions ■ ■■■ prisma/ # Database client ■ ■■■ openai/ # AI configuration ■ ■■■ utils/ # Helper functions ■■■ hooks/ # Custom React hooks ■■■ types/ # TypeScript interfaces ■■■ styles/ # Global styles, Tailwind config
```

8. Infrastructure & Deployment

Frontend Hosting

Vercel. Edge Network for global CDN. Automatic previews for PRs. Serverless functions for API routes.

Database Hosting

Railway or Supabase. Managed PostgreSQL with automatic backups. PgBouncer for connection pooling.

Cache Layer

Redis via Upstash or Railway. Session storage, rate limiting counters, API response caching.

Vector Database

Pinecone (managed) or Supabase Vector (pgvector). Semantic search for ideas and projects.

AI API

OpenAI API with tiered rate limits. Fallback to cached responses. Usage tracking per user.

Auth Service

Clerk. OAuth providers (Google, GitHub). JWT tokens with refresh rotation. User management dashboard.

Monitoring

Vercel Analytics (performance). Logtail or Datadog (logs). Sentry (error tracking).

9. Performance & Scalability

Caching Strategy

Redis 3-layer cache: L1 (API responses, 5min), L2 (User sessions, 24h), L3 (Curated ideas, 1h). Cache invalidation on data update.

Database Optimization

Connection pooling (PgBouncer, 20 max connections). Query optimization with EXPLAIN ANALYZE. Read replicas for heavy queries.

API Optimization

Lazy loading for heavy assets. Streaming responses for AI output. Batched database queries. Pagination for list endpoints.

Frontend Optimization

Next.js Image optimization. Code splitting per route. Prefetching for dashboard. Skeleton loaders for async content.

Scalability Plan

Phase 1: Single Vercel instance + Railway DB (1-10K users). Phase 2: Vercel Pro + read replicas (10-50K). Phase 3: Multi-region + CDN + auto-scaling (50K+).

10. Monitoring & Observability

Application Monitoring

Sentry for error tracking. Vercel Analytics for Web Vitals. Custom metrics: API latency, AI token usage, generation success rate.

Infrastructure Monitoring

Railway dashboard for DB health. Redis monitoring for cache hit rates. Uptime monitoring via UptimeRobot.

Business Metrics

Mixpanel or Amplitude for user behavior. Funnel analysis: Landing → Signup → Generate → Select → Export. Cohort retention analysis.

Alerting

Slack/PagerDuty integration for critical errors. Alert thresholds: API error rate > 5%, DB connection pool > 80%, AI API failures > 10%.

— End of TAD —